UNIVERSITAS BENGKULU — FAKULTAS TEKNIK

Prodi S1 Informatika | Lab. Jaringan & Sistem Tertanam

LAPORAN PRAKTIKUM LAB-2

Bus I2C, OLED SSD1306, FreeRTOS, dan Deep Sleep

Teknologi Internet of Things (IoT) — Minggu ke-3 (Bobot 2% Nilai Akhir)

Data	Anggota
Nama / NPM anggota	1. Tasya Triani G1A024015 2. Nayla Viona Azzarah G1A024045 3. Angelita Rahmatun annisa G1A024067 4. Tri Haiji Januarli G1A024069

1. Tujuan Praktikum

1. Memverifikasi keberadaan dan alamat peranti pada bus bersama I2C (GPIO 21 SDA, GPIO 22 SCL) menggunakan program I2C Scanner.
2. Mengintegrasikan sensor lingkungan BME280/BMP280 dan penampil OLED SSD1306 (128×64) secara simultan tanpa tabrakan transmisi bus.
3. Membangun arsitektur multitasking FreeRTOS dengan membagi akuisisi data dan penyegaran tampilan ke dua inti ESP32.
4. Mengonfigurasi dan menguji siklus Deep Sleep berbasis RTC Timer dan interupsi tombol EXT0, serta membuktikan persistensi variabel RTC_DATA_ATTR.

2. Dasar Teori Ringkas

2.1 Bus I2C

I2C memakai dua jalur, SDA (data) dan SCL (clock), dengan arsitektur multi-slave. Setiap slave punya alamat 7-bit unik (BME280: 0x76/0x77; SSD1306: 0x3C). Jalur bersifat open-drain sehingga membutuhkan resistor pull-up 2,2–4,7 kΩ ke 3,3 V. Pin default ESP32: SDA = GPIO 21, SCL = GPIO 22.

2.2 Sensor BME280/BMP280 dan OLED SSD1306

BME280 mengukur suhu, kelembapan, dan tekanan; BMP280 hanya suhu dan tekanan. Ketinggian diestimasi dari tekanan: h = 44330 × [1 − (P/P0)^(1/5,255)] dengan P0 = 1013,25 hPa. OLED SSD1306 memiliki GDDRAM 128×64/8 = 1024 byte; seluruh penggambaran dilakukan di buffer RAM dan baru dikirim lewat I2C ketika display.display() dipanggil (±25–30 ms pada 400 kHz).

2.3 FreeRTOS pada ESP32 dual-core

Core 0 (Protocol CPU) menangani tumpukan Wi-Fi/Bluetooth, sedangkan Core 1 (Application CPU) menjalankan setup() dan loop(). Task dibuat dengan xTaskCreatePinnedToCore() dan data dibagikan antar-task memakai mutex/queue.

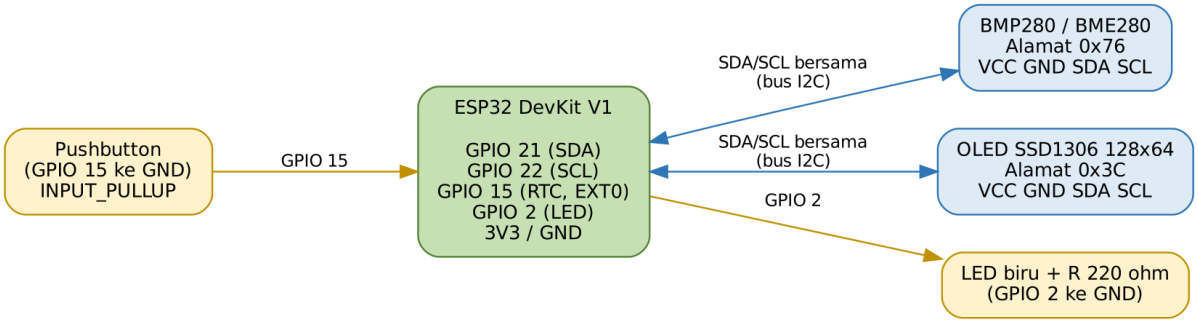
2.4 Deep Sleep dan memori RTC

Pada Deep Sleep, CPU dan SRAM utama mati (arus 10–150 µA); hanya kontroler RTC, RTC Slow Memory 8 KB, dan ULP yang hidup. Saat bangun, ESP32 reboot dari setup(). Variabel bertanda RTC_DATA_ATTR disimpan di RTC Slow Memory sehingga tetap terjaga. Sumber bangun yang dipakai: Timer (esp_sleep_enable_timer_wakeup) dan EXT0 (esp_sleep_enable_ext0_wakeup pada GPIO 15).

3. Alat dan Bahan

No	Komponen	Keterangan
1	DOIT ESP32 DevKit V1 (30-pin) + kabel Micro-USB	Mikrokontroler utama
2	Sensor BME280/BMP280	Alamat 0x76 (atau 0x77). Wokwi: wokwi-bmp280
3	OLED SSD1306 0,96 inci 128×64	Alamat 0x3C. Wokwi: board-ssd1306
4	Pushbutton	Ke GPIO 15 (RTC GPIO), pemicu EXT0, INPUT_PULLUP
5	LED biru + resistor 220 Ω	GPIO 2, indikator aktif (menyala) / tidur (mati)
6	Breadboard dan kabel jumper	Perakitan fisik
7	PC dengan PlatformIO (VS Code) atau browser untuk Wokwi, Git	Pemrograman, simulasi, dan pengumpulan kode

4. Rangkaian dan Konfigurasi



Gambar 1. Diagram koneksi bus I2C bersama, tombol EXT0, dan LED pada ESP32

Pin ESP32	Terhubung ke	Catatan
GPIO 21 (SDA)	SDA BMP280 dan SDA OLED	Bus bersama
GPIO 22 (SCL)	SCL BMP280 dan SCL OLED	Bus bersama
3V3	VCC BMP280 dan VCC OLED	Jangan ke 5V/VIN
GND	GND semua modul, tombol, dan katoda LED	Common ground

Pin ESP32	Terhubung ke	Catatan
GPIO 15	Pushbutton (sisi lain ke GND)	RTC GPIO, INPUT_PULLUP, pemicu EXT0
GPIO 2	Resistor 220 Ω lalu anoda LED	Indikator status

4.1 platformio.ini

```
[env:esp32doit-devkit-v1]
platform = espressif32
board = esp32doit-devkit-v1
framework = arduino
monitor_speed = 115200
lib_deps =
    adafruit/Adafruit Unified Sensor @ ^1.1.14
    adafruit/Adafruit BMP280 Library @ ^2.6.8
    adafruit/Adafruit BME280 Library @ ^2.2.4
    adafruit/Adafruit SSD1306 @ ^2.5.9
    adafruit/Adafruit GFX Library @ ^1.11.9
    bblanchon/ArduinoJson @ ^7.0.4
```

4.2 diagram.json (Wokwi)

```
{
  "version": 1,
  "author": "Laboratorium Informatika UNIB - Lab 2",
  "editor": "wokwi",
  "parts": [
    { "type": "wokwi-esp32-devkit-v1", "id": "esp", "top": 0, "left": 0, "attrs": {} },
    { "type": "wokwi-bmp280", "id": "bmp1", "top": -40, "left": -160, "attrs": {} },
    { "type": "board-ssd1306", "id": "oled1", "top": -40, "left": 160, "attrs": {} },
    { "type": "wokwi-pushbutton", "id": "btnWake", "top": 160, "left": -140, "attrs": { "color":
"yellow" } },
    { "type": "wokwi-led", "id": "ledStatus", "top": 140, "left": 170, "attrs": { "color": "blue" }
},
    { "type": "wokwi-resistor", "id": "rLed", "top": 185, "left": 160, "attrs": { "value": "220" }
}
  ],
  "connections": [
    [ "esp:TX0", "$serialMonitor:RX", "", [] ],
    [ "esp:RX0", "$serialMonitor:TX", "", [] ],
    [ "bmp1:VCC", "esp:3V3", "red", [] ],
    [ "bmp1:GND", "esp:GND.1", "black", [] ],
    [ "bmp1:SDA", "esp:D21", "blue", [] ],
    [ "bmp1:SCL", "esp:D22", "green", [] ],
    [ "oled1:VCC", "esp:3V3", "red", [] ],
    [ "oled1:GND", "esp:GND.1", "black", [] ],
    [ "oled1:SDA", "esp:D21", "blue", [] ],
    [ "oled1:SCL", "esp:D22", "green", [] ],
    [ "btnWake:1.l", "esp:D15", "gold", [] ],
    [ "btnWake:2.l", "esp:GND.2", "black", [] ],
```

```
    [ "esp:D2", "rLed:1", "orange", [] ],
    [ "rLed:2", "ledStatus:A", "orange", [] ],
    [ "ledStatus:C", "esp:GND.2", "black", [] ]
  ]
}
```

5. Langkah Kerja dan Eksperimen

5.1 Persiapan: Git dan proyek (25 menit)

1. Buat cabang kerja pada repositori kelompok lalu buat proyek PlatformIO/Wokwi.

```
git checkout -b feat/lab02-i2c-oled-deepsleep
```

2. Isi platformio.ini dan diagram.json seperti bagian 4.
3. Rakit rangkaian sesuai Gambar 1 (SDA/SCL bersama, VCC ke 3V3, tombol di GPIO 15, LED di GPIO 2).

5.2 Eksperimen 1: Diagnostik I2C (40 menit)

1. Unggah program I2C Scanner berikut, buka Serial Monitor 115200 baud.

```
#include <Arduino.h>
#include <Wire.h>

void setup() {
  Serial.begin(115200);
  Wire.begin(21, 22);   // SDA = GPIO 21, SCL = GPIO 22
  Serial.println("\n--- Memindai Peranti pada Bus I2C ESP32 ---");
  byte hitung = 0;
  for (byte alamat = 1; alamat < 127; alamat++) {
    Wire.beginTransmission(alamat);
    byte statusGalat = Wire.endTransmission();
    if (statusGalat == 0) {
      Serial.printf("Peranti I2C ditemukan pada alamat: 0x%02X\n", alamat);
      hitung++;
    }
  }
  Serial.printf("Pemindaian selesai. Total peranti ditemukan: %d\n", hitung);
}
void loop() {}
```

2. Catat alamat yang muncul pada Tabel 3.1 (diharapkan 0x3C dan 0x76).
3. Bila peranti tidak terdeteksi: periksa kabel SDA/SCL, VCC 3V3, GND bersama, dan pull-up.
4. Lanjutkan dengan membaca suhu, tekanan, dan elevasi BMP/BME280 pada Serial Monitor (kode lengkap pada Eksperimen 2).

5.3 Eksperimen 2: FreeRTOS Dual-Task (45 menit)

1. Pada program terpadu (Lampiran A), sensor dibaca di setup() yang berjalan pada Core 1.
2. Task TaskTampilanDanSerial dibuat dengan xTaskCreatePinnedToCore(..., 4096, NULL, 1, NULL, 0) sehingga berjalan pada Core 0: menampilkan data ke OLED, membuat JSON ke Serial Monitor, dan menyimpan riwayat ke memori RTC.
3. Data berpindah antar-inti lewat struct dataCuaca yang dilindungi mutexData (xSemaphoreTake/Give).
4. Analisis latensi: ukur durasi pengiriman layar dengan potongan kode berikut lalu catat pada Tabel 3.3.

```
uint32_t t0 = micros();
display.display();                    // kirim 1024 byte via I2C
uint32_t dt = micros() - t0;
Serial.printf("display.display(): %u us\n", dt);
```

Tabel 3.3 Pengukuran latensi (pendukung analisis)

Pengukuran	Hasil Anda	Nilai acuan modul
Durasi display.display() (I2C 400 kHz)	…… ms	±25–30 ms
Durasi pembacaan sensor (takeForcedMeasurement + baca)	…… ms	—
Waktu total setup() sampai Deep Sleep	…… ms	±1 detik (≈ 500 ms delay + init)

5.4 Eksperimen 3: Deep Sleep

1. Program mengatur Timer wakeup 10 detik dan EXT0 pada GPIO 15 (LOW). LED GPIO 2 menyala saat aktif dan mati sesaat sebelum esp_deep_sleep_start().
2. Amati lima siklus: Boot 1 (power-on), Boot 2–3 (Timer), Boot 4 (tekan tombol GPIO 15 saat modul tidur), Boot 5 (Timer).
3. Catat nomor siklus, sumber pemicu, suhu, tekanan, elevasi, dan riwayat RTC siklus lalu pada Tabel 3.2.
4. Pada Wokwi, ubah nilai suhu/tekanan sensor di antara siklus agar perubahan riwayat RTC terlihat jelas.

5.5 Verifikasi dan pengumpulan (20 menit)

1. Demonstrasikan unjuk kerja kepada asisten dan minta paraf pada ceklis (bagian 7).
2. Commit dan push kode ke GitHub kelompok:

```
git add .
git commit -m "Lab-2: I2C, OLED SSD1306, FreeRTOS, Deep Sleep"
git push -u origin feat/lab02-i2c-oled-deepsleep
```

6. Hasil Pengamatan

Tabel 3.1 Hasil diagnostik pemindaian alamat bus I2C

No	Nama komponen peranti	Alamat heksadesimal	Keterangan fungsi pada sistem
1	Sensor tekanan/suhu BMP280 / BME280	0x76 (atau 0x77 bila SDO ke VCC)	Mengukur suhu dan tekanan; dasar perhitungan elevasi
2	Display grafis OLED SSD1306	0x3C (atau 0x3D)	Menampilkan suhu, tekanan, elevasi, nomor siklus, dan pemicu

Keluaran Serial Monitor:

```
Peranti I2C ditemukan pada alamat: 0x3C
Peranti I2C ditemukan pada alamat: 0x76
Pemindaian selesai. Total peranti ditemukan: 2
```

Keterangan: kedua alamat berbeda (0x3C dan 0x76) sehingga tidak terjadi tabrakan alamat pada bus bersama.

Tabel 3.2 Pengamatan siklus bangun-tidur (duty cycling) dan memori RTC

CONTOH ILUSTRASI (berdasarkan nilai bawaan Wokwi: ±24 °C, ±1013 hPa). GANTI dengan hasil pengamatan Anda.

Siklus	Sumber pemicu	Suhu (°C)	Tekanan (hPa)	Elevasi (m)	Nilai riwayat siklus lalu di RTC
Boot 1	Power-on Reset	24,00	1013,00	≈ 2	Suhu 0,00 (belum ada riwayat)
Boot 2	Timer (10 s)	24,00	1013,00	≈ 2	Suhu 24,00

Siklus	Sumber pemicu	Suhu (°C)	Tekanan (hPa)	Elevasi (m)	Nilai riwayat siklus lalu di RTC
Boot 3	Timer (10 s)	24,00	1013,00	≈ 2	Suhu 24,00
Boot 4	Tombol EXT0 (GPIO 15)	24,00	1013,00	≈ 2	Suhu 24,00
Boot 5	Timer (10 s)	24,00	1013,00	≈ 2	Suhu 24,00

Interpretasi hasil: (1) nomor siklus naik 1 hingga 5 tanpa kembali ke 1 sehingga RTC_DATA_ATTR terbukti bertahan selama Deep Sleep; (2) riwayat suhu hanya 0,00 pada Boot 1 karena memori RTC baru terisi setelah siklus pertama; (3) Boot 4 menunjukkan pemicu EXT0 karena tombol ditekan sebelum timer 10 detik berakhir; (4) elevasi ±2 m konsisten dengan tekanan 1013 hPa terhadap P0 = 1013,25 hPa.

7. Pertanyaan Analisis Laporan (Level C1–C6)

C1 (Mengingat)

Pertanyaan: Berapakah konsumsi arus ESP32 saat berada di mode aktif penuh dibandingkan saat berada di mode Deep Sleep? Komponen perangkat keras apa saja yang dimatikan pada mode Deep Sleep?

Jawaban:

Mode	Arus tipikal	Keterangan
Active (aktif penuh)	80–240 mA	CPU ganda, radio Wi-Fi/BT, dan semua periferal aktif
Deep Sleep	10–150 µA	Hanya RTC controller dan ULP yang hidup

Selisihnya ribuan kali lipat (80 mA / 150 µA ≈ 533× hingga 240 mA / 10 µA = 24.000×).

- Dimatikan saat Deep Sleep: kedua inti CPU, SRAM utama 520 KB, modul radio Wi-Fi/Bluetooth, dan periferal digital.
- Tetap hidup: kontroler RTC, RTC Slow Memory 8 KB, koprosesor ULP, dan pin RTC GPIO (untuk pemicu bangun).

C2 (Memahami)

Pertanyaan: Mengapa variabel global biasa selalu tereset ke nol saat ESP32 terbangun dari Deep Sleep, sedangkan variabel dengan atribut RTC_DATA_ATTR dapat mempertahankan nilainya? Di area memori fisik mana variabel tersebut disimpan?

Jawaban:

- Variabel global biasa berada di SRAM utama. Saat Deep Sleep, SRAM diputus dayanya sehingga isinya hilang.
- Setelah bangun, ESP32 reboot dari awal; startup code menginisialisasi ulang variabel global ke nilai awal (0), sehingga hitungan seperti int counter = 0 selalu kembali ke 0.
- Variabel RTC_DATA_ATTR ditempatkan linker pada section .rtc.data di **RTC Slow Memory (8 KB)**, bagian domain daya RTC yang tetap dialiri daya selama tidur. Karena itu nilainya bertahan antar siklus.
- Batasan: data RTC hilang pada power-on reset penuh atau saat pin EN/CHIP_PU ditekan, karena domain RTC ikut mati.

C3 (Menerapkan)

Pertanyaan: Jika stasiun cuaca tidur Deep Sleep 15 menit (t_tidur = 899 detik, I_tidur = 40 μA) dan aktif membaca sensor serta menampilkan data selama 1 detik (I_aktif = 85 mA), hitung I_avg dan estimasi masa pakai baterai Li-Ion 3000 mAh (η = 0,80) dalam bulan.

Jawaban:

> Diketahui: t_tidur = 899 s; I_tidur = 40 μA = 0,040 mA
> t_aktif = 1 s; I_aktif = 85 mA
> Baterai 3000 mAh; efisiensi η = 0,80
>
> Periode siklus T = 1 + 899 = 900 s (= 15 menit)
>
> I_avg = (I_aktif x t_aktif + I_tidur x t_tidur) / T
> = (85 x 1 + 0,040 x 899) / 900
> = (85 + 35,96) / 900
> = 120,96 / 900
> = 0,1344 mA
>
> Kapasitas efektif = 3000 x 0,80 = 2400 mAh
> Masa pakai (jam) = 2400 / 0,1344 = 17.857 jam
> Masa pakai (hari) = 17.857 / 24 = 744 hari
> Masa pakai (bulan)= 744 / 30,4 = 24,5 bulan

Hasil: I_avg ≈ 0,134 mA dan masa pakai ≈ 24,5 bulan (sekitar 2 tahun). Dari pengamatan, waktu aktif nyata ±1 detik (±500 ms delay ditambah inisialisasi), sesuai asumsi soal. Estimasi ideal: self-discharge baterai serta arus regulator dan modul sensor belum dihitung, sehingga masa pakai riil lebih pendek. Sekitar 70% energi terpakai pada fase aktif (85 dari 120,96 mA·s) walau hanya 0,11% waktu.

C4 (Menganalisis)

Pertanyaan: Pada Listing Program 3, TaskTampilanDanSerial dipin ke Core 0 dengan xTaskCreatePinnedToCore() sementara setup() berjalan di Core 1. Analisislah keuntungan arsitektur ini terhadap pencegahan pemblokiran pembacaan sensor oleh transmisi data grafis I2C yang lambat.

Jawaban:

- Masalah dasar. Pengiriman 1024 byte buffer ke OLED melalui I2C 400 kHz membutuhkan ±25–30 ms (belum termasuk waktu menggambar teks dan garis). Bila dikerjakan berurutan dengan pembacaan sensor di satu alur, sensor harus menunggu selesainya transfer layar.
- Pemisahan beban. Task OLED+JSON dipin ke Core 0, sedangkan setup()/loop() di Core 1. Kedua inti berjalan paralel sehingga pembacaan BMP/BME280 dan logika utama di Core 1 tidak tertahan oleh transmisi grafis.
- Sinkronisasi data. Struct dataCuaca dilindungi mutexData, sehingga task UI tidak membaca data setengah-tertulis (race condition) saat sensor menulis nilai baru.
- Penjadwalan. Task memakai prioritas 1 dan, pada versi periodik, vTaskDelay (non-blocking) sehingga CPU dilepas ke task lain selama menunggu.
- Isolasi fungsi. Core 0 (Protocol CPU) umumnya mengurus Wi-Fi/Bluetooth. Pada Lab-2 radio tidak dipakai, jadi beban UI di Core 0 aman; bila Wi-Fi diaktifkan, task berat di Core 0 perlu dibatasi agar tidak mengganggu tumpukan jaringan.

Alur waktu singkat

Waktu	Core 1 (setup)	Core 0 (TaskUI)
0 ms	Mulai, init I2C/OLED/sensor	Belum ada
≈ t1	Baca sensor, isi dataCuaca (mutex)	Task dibuat

Waktu	Core 1 (setup)	Core 0 (TaskUI)
t1 – t1+500 ms	delay(500): menunggu	Ambil data, gambar OLED, kirim I2C (±25–30 ms), cetak JSON, simpan RTC
t1+500 ms	Set wakeup, LED mati, deep sleep	Selesai: vTaskDelete(NULL)

Catatan kritis: kedua inti tetap berbagi satu bus I2C fisik dan pustaka Wire tidak thread-safe. mutexData hanya melindungi struct data, bukan bus. Pada Listing 3 aman karena sensor selesai dibaca di setup() sebelum task OLED dijalankan; bila sensor dibaca berkala sementara OLED diperbarui, perlu mutex khusus bus I2C. Selain itu delay(500) sebagai penunggu task rapuh; alternatif yang lebih baik adalah event group atau task notification.

C5 (Mengevaluasi)

Pertanyaan: Bandingkan kelebihan dan kelemahan pemicu bangun EXT0 (pin tunggal) terhadap EXT1 (multi-pin) untuk stasiun hidrologi deteksi banjir dengan 3 sensor ambang tinggi muka air terpisah.

Jawaban:

Aspek	EXT0	EXT1
Jumlah pin	1 pin RTC GPIO	Banyak pin RTC GPIO (bitmask)
Logika pemicu	Satu level (HIGH atau LOW)	ANY_HIGH atau ALL_LOW
Identifikasi pemicu	Tidak perlu (hanya 1 pin)	Bisa dibaca via esp_sleep_get_ext1_wakeup_status()
Implementasi 3 sensor	Harus digabung dengan gerbang OR/dioda eksternal; sensor mana yang aktif tidak diketahui	3 pin langsung; ambang yang tercapai dapat diidentifikasi
Kelebihan	Sederhana, kode pendek, andal untuk 1 tombol/sensor	Tanpa gerbang tambahan; mendukung multi-ambang bertingkat
Kelemahan	Perlu perangkat keras tambahan untuk 3 sensor dan kehilangan identitas ambang	Konfigurasi lebih kompleks; semua pin harus RTC GPIO; pull-up/pull-down internal tidak andal sehingga disarankan resistor eksternal

Kesimpulan evaluasi: untuk tiga ambang (siaga, waspada, awas), **EXT1 mode ANY_HIGH lebih tepat** karena tiap sensor masuk langsung dan tingkat bahaya dapat diketahui dari bitmask saat bangun. Contoh implementasi:

```
#define BM_SIAGA    (1ULL << 32)   // GPIO 32
#define BM_WASPADA  (1ULL << 33)   // GPIO 33
#define BM_AWAS     (1ULL << 34)   // GPIO 34 (input-only, pull-down eksternal)

esp_sleep_enable_ext1_wakeup(BM_SIAGA | BM_WASPADA | BM_AWAS,
                             ESP_EXT1_WAKEUP_ANY_HIGH);

// Setelah bangun:
if (esp_sleep_get_wakeup_cause() == ESP_SLEEP_WAKEUP_EXT1) {
  uint64_t pin = esp_sleep_get_ext1_wakeup_status();
  if (pin & BM_AWAS)        { /* AWAS: kirim peringatan tertinggi */ }
  else if (pin & BM_WASPADA) { /* WASPADA */ }
  else if (pin & BM_SIAGA)   { /* SIAGA  */ }
}
```

Gunakan resistor pull-down eksternal (mis. 10 kΩ) pada ketiga pin agar tidak mengambang saat tidur.

C6 (Mencipta): Penyimpan Ekstrem Cuaca

Pertanyaan: Modifikasi main.cpp agar menyimpan suhu tertinggi (suhuMaks) dan terendah (suhuMin) sepanjang sejarah operasional alat di memori RTC, lalu tampilkan rekor tersebut pada baris bawah layar OLED setiap bangun. Lampirkan kode dan tangkapan layar bukti.

Rancangan:

- Dua variabel baru RTC_DATA_ATTR: suhuMaks (awal −999) dan suhuMin (awal 999) sebagai sentinel agar pembacaan pertama langsung menjadi rekor.
- Pada TaskTampilanDanSerial, nilai rekor diperbarui dari pembacaan valid sebelum layar digambar.
- Fungsi perbaruiTampilanOled() diberi parameter rekor dan tata letak digeser agar muat 6 baris: judul, suhu, tekanan, elevasi, rekor (Mx/Mn), dan status siklus.
- Rekor juga ditambahkan ke JSON Serial (temp_max_c dan temp_min_c).

Potongan kode perubahan:

```
// 1) Variabel RTC baru
RTC_DATA_ATTR float suhuMaks = -999.0f;   // sentinel: belum ada data
RTC_DATA_ATTR float suhuMin  =  999.0f;

// 2) Dalam TaskTampilanDanSerial, di dalam if (lokal.valid), sebelum menggambar:
if (lokal.suhu > suhuMaks) suhuMaks = lokal.suhu;
if (lokal.suhu < suhuMin)  suhuMin  = lokal.suhu;

// 3) Tampilan baris rekor pada perbaruiTampilanOled():
display.setCursor(0, 41);
display.printf("Mx:%.1f Mn:%.1f C", tMaks, tMin);
```

Kode lengkap main.cpp hasil modifikasi dilampirkan pada Lampiran A. Contoh hasil yang diharapkan: pada Boot 1 layar menampilkan Mx:24.0 Mn:24.0; setelah suhu sensor Wokwi dinaikkan ke 30 °C pada siklus berikutnya menjadi Mx:30.0 Mn:24.0, dan nilai itu bertahan walau modul tidur lagi.

Catatan: *Adafruit_BMP280::begin(0x76) memeriksa chip ID 0x58. Jika sensor Anda sebenarnya BME280 (ID 0x60), inisialisasi gagal; gunakan pustaka Adafruit_BME280 atau bmp.begin(0x76, 0x60).*

8. Kesimpulan

1. Dua peranti I2C (BMP280/BME280 di 0x76 dan OLED SSD1306 di 0x3C) dapat berbagi satu bus (GPIO 21/22) tanpa tabrakan karena alamatnya berbeda dan pull-up tersedia; I2C Scanner efektif untuk verifikasi.
2. FreeRTOS memungkinkan pemisahan akuisisi sensor (Core 1) dari visualisasi OLED dan serialisasi JSON (Core 0), sehingga transfer grafis I2C ±25–30 ms tidak menahan pembacaan sensor. Akses bus I2C bersama tetap perlu mutex bila dipakai lintas-task.
3. Deep Sleep 10 detik dengan pemicu Timer dan EXT0 berhasil; arus turun ke orde puluhan hingga ratusan µA. Pada contoh perhitungan 15 menit, arus rata-rata ≈ 0,134 mA dan baterai 3000 mAh bertahan ≈ 24,5 bulan.
4. RTC_DATA_ATTR membuktikan data (nomor siklus, riwayat, dan rekor suhu) dapat bertahan melewati reboot Deep Sleep, sedangkan variabel biasa tereset.

Daftar Pustaka

1. Bahan Ajar Minggu 3 Teknologi IoT, Prodi S1 Informatika FT UNIB.
2. Panduan Praktikum Minggu 3 (Lab-2), Lab. Jaringan & Sistem Tertanam FT UNIB.

3. Espressif Systems, ESP32 Technical Reference Manual: I2C Controller, Low-Power Management & RTC.
4. Cameron, N. (2023), ESP32 Formats and Communication, Apress, Bab 2–4.
5. Lea, P. (2020), IoT and Edge Computing for Architects, Packt, Bab 4.
6. Hamdan et al. (2020), Sensors, 20(22), 6441.

Lampiran A. Kode Lengkap main.cpp (Hasil Modifikasi C6)

Berdasarkan Listing 3 pada panduan praktikum, dengan penambahan suhuMaks/suhuMin pada memori RTC dan baris rekor pada OLED.

```
#include <Arduino.h>
#include <Wire.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_BMP280.h>
#include <Adafruit_SSD1306.h>
#include <ArduinoJson.h>

#define LEBAR_LAYAR 128
#define TINGGI_LAYAR 64
#define ALAMAT_OLED 0x3C
#define PIN_LED_STATUS 2
#define PIN_WAKEUP_TOMBOL GPIO_NUM_15
#define DURASI_TIDUR_DETIK 10

Adafruit_SSD1306 display(LEBAR_LAYAR, TINGGI_LAYAR, &Wire, -1);
Adafruit_BMP280 bmp;

// --- VARIABEL PERSISTEN DI RTC SLOW MEMORY ---
RTC_DATA_ATTR int   nomorSiklusBoot = 0;
RTC_DATA_ATTR float riwayatSuhuTerakhir = 0.0f;
RTC_DATA_ATTR float riwayatTekananTerakhir = 0.0f;
RTC_DATA_ATTR float suhuMaks = -999.0f;   // [C6] rekor suhu tertinggi
RTC_DATA_ATTR float suhuMin  =  999.0f;   // [C6] rekor suhu terendah

struct DataSensor {
```

```
  float suhu;
  float tekanan;
  float elevasi;
  bool valid;
} dataCuaca;

SemaphoreHandle_t mutexData;

void cetakDiagnostikWakeup() {
  esp_sleep_wakeup_cause_t alasan = esp_sleep_get_wakeup_cause();
  Serial.printf("\n=======================================\n");
  Serial.printf("[RTC] Siklus Bangun Ke-%d\n", nomorSiklusBoot);
  switch (alasan) {
    case ESP_SLEEP_WAKEUP_TIMER:
      Serial.println("[RTC WAKEUP] Pemicu: Pewaktu Internal (RTC Timer)");
      break;
    case ESP_SLEEP_WAKEUP_EXT0:
      Serial.println("[RTC WAKEUP] Pemicu: Tombol Darurat Luar (EXT0 pada GPIO 15)");
      break;
    default:
      Serial.println("[RTC WAKEUP] Pemicu: Power-on Reset / Boot Pertama");
      break;
  }
  Serial.printf("[RTC MEM] Riwayat Siklus Lalu -> Suhu: %.2f C | Tekanan: %.2f hPa\n",
                riwayatSuhuTerakhir, riwayatTekananTerakhir);
  Serial.printf("[RTC MEM] Rekor Suhu -> Maks: %.2f C | Min: %.2f C\n", suhuMaks,
suhuMin);
}

// [C6] ditambah parameter tMaks dan tMin; tata letak digeser
void perbaruiTampilanOled(float t, float p, float alt, int siklus,
                          const char* pemicu, float tMaks, float tMin) {
  display.clearDisplay();
  display.setTextSize(1);
```

```
  display.setTextColor(SSD1306_WHITE);

  display.setCursor(0, 0);
  display.printf("UNIB IoT - STASIUN CUACA");
  display.drawLine(0, 9, 127, 9, SSD1306_WHITE);

  display.setCursor(0, 11);  display.printf("Suhu    : %.1f C", t);
  display.setCursor(0, 20);  display.printf("Tekanan : %.1f hPa", p);
  display.setCursor(0, 29);  display.printf("Elevasi : %.1f m", alt);

  display.drawLine(0, 38, 127, 38, SSD1306_WHITE);
  display.setCursor(0, 41);  display.printf("Mx:%.1f Mn:%.1f C", tMaks, tMin);

  display.drawLine(0, 51, 127, 51, SSD1306_WHITE);
  display.setCursor(0, 55);  display.printf("#%d | %s", siklus, pemicu);
  display.display();
}

// Task FreeRTOS pada Core 0: UI dan serialisasi
void TaskTampilanDanSerial(void *pvParameters) {
  xSemaphoreTake(mutexData, portMAX_DELAY);
  DataSensor lokal = dataCuaca;
  xSemaphoreGive(mutexData);

  if (lokal.valid) {
    esp_sleep_wakeup_cause_t alasan = esp_sleep_get_wakeup_cause();
    const char* labelPemicu = (alasan == ESP_SLEEP_WAKEUP_EXT0) ? "EXT0 BTN" :
"TIMER";

    // [C6] perbarui rekor sebelum menggambar
    if (lokal.suhu > suhuMaks) suhuMaks = lokal.suhu;
    if (lokal.suhu < suhuMin)  suhuMin  = lokal.suhu;

    // 1. Tampilkan ke OLED
```

```
    perbaruiTampilanOled(lokal.suhu, lokal.tekanan, lokal.elevasi,
                         nomorSiklusBoot, labelPemicu, suhuMaks, suhuMin);

    // 2. Serialisasi JSON
    JsonDocument doc;
    doc["device_id"]    = "esp32-weather-st01";
    doc["boot_count"]   = nomorSiklusBoot;
    doc["wakeup_cause"] = labelPemicu;
    doc["temp_c"]       = serialized(String(lokal.suhu, 2));
    doc["pressure_hpa"] = serialized(String(lokal.tekanan, 2));
    doc["altitude_m"]   = serialized(String(lokal.elevasi, 1));
    doc["prev_temp_c"]  = serialized(String(riwayatSuhuTerakhir, 2));
    doc["temp_max_c"]   = serialized(String(suhuMaks, 2));   // [C6]
    doc["temp_min_c"]   = serialized(String(suhuMin, 2));    // [C6]
    serializeJson(doc, Serial);
    Serial.println();

    // Simpan ke memori RTC untuk siklus berikutnya
    riwayatSuhuTerakhir = lokal.suhu;
    riwayatTekananTerakhir = lokal.tekanan;
  }
  vTaskDelete(NULL);   // task selesai
}

void setup() {
  Serial.begin(115200);
  pinMode(PIN_LED_STATUS, OUTPUT);
  pinMode(PIN_WAKEUP_TOMBOL, INPUT_PULLUP);
  digitalWrite(PIN_LED_STATUS, HIGH);   // LED menyala: fase aktif

  nomorSiklusBoot++;
  cetakDiagnostikWakeup();

  mutexData = xSemaphoreCreateMutex();
```

```
  Wire.begin(21, 22);

  if (!display.begin(SSD1306_SWITCHCAPVCC, ALAMAT_OLED)) {
    Serial.println("Gagal menemukan layar OLED SSD1306!");
  }

  if (!bmp.begin(0x76)) {
    Serial.println("Gagal mendeteksi sensor BMP/BME280 pada alamat 0x76!");
    dataCuaca.valid = false;
  } else {
    bmp.setSampling(Adafruit_BMP280::MODE_FORCED,
                    Adafruit_BMP280::SAMPLING_X2,
                    Adafruit_BMP280::SAMPLING_X16,
                    Adafruit_BMP280::FILTER_X16,
                    Adafruit_BMP280::STANDBY_MS_500);
    bmp.takeForcedMeasurement();

    xSemaphoreTake(mutexData, portMAX_DELAY);
    dataCuaca.suhu    = bmp.readTemperature();
    dataCuaca.tekanan = bmp.readPressure() / 100.0F;   // Pa -> hPa
    dataCuaca.elevasi = bmp.readAltitude(1013.25);
    dataCuaca.valid   = true;
    xSemaphoreGive(mutexData);
  }

  // Task UI pada Core 0 (Protocol Core)
  xTaskCreatePinnedToCore(TaskTampilanDanSerial, "TaskUI", 4096, NULL, 1, NULL, 0);

  delay(500);   // beri waktu Task UI menyelesaikan tugas

  // --- PERSIAPAN MASUK DEEP SLEEP ---
  Serial.printf("[POWER] Menyiapkan Deep Sleep selama %d detik...\n",
DURASI_TIDUR_DETIK);
  esp_sleep_enable_timer_wakeup((uint64_t)DURASI_TIDUR_DETIK * 1000000ULL);
```

```
  esp_sleep_enable_ext0_wakeup(PIN_WAKEUP_TOMBOL, 0);    // bangun jika GPIO 15
LOW

  digitalWrite(PIN_LED_STATUS, LOW);   // LED mati: memasuki Deep Sleep
  Serial.println("[POWER] Zzz... Memasuki Deep Sleep. Arus turun ke ~10-150 uA.");
  Serial.flush();
  esp_deep_sleep_start();
}

void loop() {
  // Kosong: sistem bekerja pada arsitektur Deep Sleep penuh
}
```